Guía del laboratorio — OWASP Amass

Laboratorio de reconocimiento y mapeo de superficie de ataque
Dominio ficticio: `lab.local` · Red: `192.168.56.0/24`

Expositor: Michael Stiven Tabares Tobón
Curso: Seguridad de la Información — 2026-II · Ingeniería de Sistemas, UdeA

Las dos "máquinas" del laboratorio

Aunque los dos corren en Docker, piénsalos como dos máquinas separadas:

Máquina	Cómo se llama	Qué hace	Cómo entras
Tu portátil (Arch)	`host`	Tu equipo. Aquí escribes los comandos.	Ya estás aquí
Servidor DNS	`dns`	Sirve la zona ficticia `lab.local`	`docker compose exec dns ..` .
Amass	`amass`	La herramienta. Aquí corres los comandos de Amass.	`docker compose exec amass bash`

> **Regla:** todo comando de `amass` se ejecuta **dentro de la máquina `amass`**.
> Todo comando de `dig` / `nslookup` se puede ejecutar dentro de `amass` o en `dns`.

Índice

Para operar el laboratorio (usa el script)
- Paso 0 (opcional) · Instalar Amass nativo
- Paso 1 · Montar el laboratorio
- Paso 2 · Entrar dentro de la máquina Amass
- Paso 3 · Comandos de la práctica
- Paso 4 · Experimentos para aprender
- Paso 5 · Resumen: dónde se ejecuta cada cosa

Referencia
- Los atajos del script
- La zona del laboratorio
- Problemas frecuentes
- Dos detalles técnicos

Paso 0 (opcional) · Instalar Amass nativo en tu equipo

> **¿Hace falta para el laboratorio? No.** El laboratorio ya trae Amass dentro del contenedor Docker y `./start.sh` se encarga de todo.
>
> Esta sección es para cuando quieras **aprender Amass en tu propia máquina**, fuera del laboratorio. Ten en cuenta que Amass instalado de forma nativa **sí sale a Internet**: consulta fuentes OSINT públicas (crt.sh, RapidDNS, HackerTarget...) y resuelve con servidores DNS públicos.

En Arch Linux

Opción A — Binario oficial (rápido, sin compilar)

```
cd /tmp
curl -LO https://github.com/owasp-amass/amass/releases/download/v5.1.3/amass_linux_amd64.tar.gz
curl -LO https://github.com/owasp-amass/amass/releases/download/v5.1.3/amass_checksums.txt

# Verificar la integridad del archivo
sha256sum -c amass_checksums.txt --ignore-missing

# Descomprimir e instalar
tar xzf amass_linux_amd64.tar.gz
sudo install -m 755 amass_linux_amd64/amass /usr/local/bin/amass

# Limpiar
rm -rf amass_linux_amd64 amass_linux_amd64.tar.gz amass_checksums.txt
```

> En ARM (Raspberry Pi, laptops ARM) cambia `amass_linux_amd64.tar.gz` por `amass_linux_arm64.tar.gz`.

Alternativa sin sudo (instalar en tu carpeta personal)

```
mkdir -p ~/.local/bin
cd /tmp
curl -LO https://github.com/owasp-amass/amass/releases/download/v5.1.3/amass_linux_amd64.tar.gz
tar xzf amass_linux_amd64.tar.gz
install -m 755 amass_linux_amd64/amass ~/.local/bin/amass

# Asegura que ~/.local/bin está en el PATH
grep -q 'HOME/.local/bin' ~/.bashrc || \
  echo 'export PATH="$HOME/.local/bin:$PATH"' >> ~/.bashrc
source ~/.bashrc

# Comprobar
which amass && amass -version
```

> `~/.local/bin` ya viene en el `PATH` por defecto en Arch. Si `which amass` no lo encuentra tras instalar, es que tu `~/.bashrc` no lo incluye: agrega la línea del `export` de arriba.

Opción B — AUR

```
# Con yay (o cualquier helper de AUR)
yay -S amass-bin

# O manualmente con makepkg
git clone https://aur.archlinux.org/amass-git.git
cd amass-git
makepkg -si
```

Opción C — Compilar desde el código (necesita Go 1.25+)

```
sudo pacman -S --needed go git
git clone https://github.com/owasp-amass/amass.git
cd amass
CGO_ENABLED=0 go install -v github.com/owasp-amass/amass/v5/cmd/amass@main
```

El binario queda en `$(go env GOPATH)/bin`. Para que esté en el `PATH`:

```
echo 'export PATH="$PATH:$(go env GOPATH)/bin"' >> ~/.bashrc
source ~/.bashrc
```

En Kali Linux

Kali trae Amass en sus repositorios:

```
sudo apt update
sudo apt install -y amass
```

> **Atención:** la versión de los repositorios de Kali puede no ser la última. Para instalar la v5 concreta, usa el binario oficial o compila desde el código (las mismas órdenes de Arch, Opciones A y C).

Alternativa universal: Docker

Si no quieres instalar nada en el sistema:

```
docker pull owaspamass/amass:latest
docker run --rm owaspamass/amass:latest -version
```

Confirmar que quedó instalado

```
# 1. ¿Está en el PATH?
which amass
# Esperado: /usr/local/bin/amass   (Arch)
#           /usr/bin/amass           (Kali)

# 2. ¿Qué versión es?
amass -version
# Esperado: v5.1.3

# 3. ¿Responde la ayuda?
amass -h
# Debe listar los subcomandos: assoc, engine, enum, subs, track, viz

# 4. ¿Qué subcomandos tiene?
amass -h 2>&1 | grep -oE "\[assoc\|engine\|enum\|subs\|track\|viz\]"
# Esperado: [assoc|engine|enum|subs|track|viz]

# 5. Comprobación final
amass -version && echo "INSTALACIÓN CORRECTA"
```

> **Sobre `amass enum -list`:** en la v5 este comando **no** solo lista las fuentes: además de eso **ejecuta una enumeración** y por eso pide un dominio (`-d`) y tarda. Úsalo solo si quieres ver la lista completa de fuentes activas: `amass enum -list -d lab.local`. Para una comprobación rápida, `amass -h` es suficiente.

Si `which amass` no devuelve nada, el binario no está en el `PATH`. En Arch suele estar en `/usr/local/bin` (revisa con `echo $PATH`); en Kali, en `/usr/bin`.

Paso 1 · Montar el laboratorio

Desde tu portátil (Arch), en la raíz del proyecto:

```
./start.sh
```

Eso construye todo, levanta los contenedores y comprueba que el DNS responde.
Tarda ~2 minutos la primera vez.

Salida esperada:

```
==> Verificando requisitos
 [✓] Docker: Docker version 29.7.2
 [✓] Docker Compose: 5.5.1
 [✓] Daemon de Docker accesible

==> Construyendo las imágenes (la primera vez tarda ~2 min)
 [✓] Imágenes construidas

==> Levantando el laboratorio
 [✓] Contenedores iniciados

==> Esperando a que el motor de Amass esté listo
 [✓] Motor listo tras 1s

==> Verificando el DNS del laboratorio
 [✓] DNS del laboratorio responde (192.168.56.101:53)
 [✓] Resolución de subdominios funciona (dev.lab.local → 192.168.56.103)
 [✓] NXDOMAIN correcto para los candidatos inexistentes
```

Para empezar de cero en cualquier momento:

```
./start.sh --reset
```

Paso 2 · Entrar dentro de la máquina Amass

Esta es la orden clave. Desde tu portátil, en la carpeta `lab-infra/docker`:

```
cd lab-infra/docker
docker compose -f compose.yaml exec amass bash
```

Ahora tienes una **terminal dentro de la máquina** `amass`. El prompt cambia a algo como:

```
/ # estás dentro del contenedor
```

Todo lo que hagas de aquí en adelante corre dentro de Amass. Para salir:

```
exit
```

> **Atajo:** si no quieres entrar y salir, puedes ejecutar un solo comando directamente:
>
> ```
> docker compose -f compose.yaml exec amass amass subs -names -d lab.local -ip
> ```

Verificar que estás dentro

```
hostname          # deve mostrar algo como amass-lab-amass
which amass      # /bin/amass
```

Paso 3 · Comandos de la práctica

3.1 Ver la versión de Amass

> 📍 **Dónde:** dentro de la máquina `amass`

```
amass -version
```

```
v5.1.3
```

3.2 Ver la ayuda de cada subcomando

> 📍 **Dónde:** dentro de la máquina `amass`

```
amass -h              # ayuda general
amass enum -h         # ayuda de enum
amass subs -h         # ayuda de subs
```

3.3 Comprobar que el DNS ficticio responde

> 📍 **Dónde:** dentro de la máquina `amass`

```
dig +short @192.168.56.101 lab.local A
```

```
192.168.56.101
```

Un subdominio que sí existe:

```
dig +short @192.168.56.101 dev.lab.local A
```

```
192.168.56.103
```

Uno que **no** existe:

```
dig +short @192.168.56.101 test.lab.local A
```

```
(sin salida — el DNS responde NXDOMAIN)
```

> Esto último es importante: **el DNS es el que decide qué existe.** Lo volveremos a ver en la fuerza bruta.

3.4 La enumeración con fuerza bruta

> 📍 **Dónde:** dentro de la máquina `amass`

```
amass enum \
  -engine http://127.0.0.1:4000 \
  -active \
  -brute -w /wl/subdomains-lab.txt \
  -d lab.local \
  -p 80,443 \
  -timeout 3
```

Qué significa cada parte:

Parte	En palabras
`amass enum`	"Haz una enumeración"
`-engine http://127.0.0.1:4000`	"Habla con el motor que corre en esta misma máquina"
`-active`	"Pregunta al DNS y comprueba los puertos"
`-brute -w /wl/subdomains-lab.txt`	"Prueba los nombres de esta lista"
`-d lab.local`	"El objetivo es este dominio"
`-p 80,443`	"Solo mira estos dos puertos"
`-timeout 3`	"Si dejas de avanzar 3 minutos, te detienes"

Tarda entre 2 y 4 minutos. Verás una barra de progreso.

> ⚠ **No uses** `-nf` : en Amass v5.1.3 esa opción provoca un error interno que tumba el motor. La fuerza bruta se hace con `-brute -w`.

3.5 Ver los resultados

> 📍 **Dónde:** dentro de la máquina `amass`

```
amass subs -names -d lab.local -ip
```

```
mail.lab.local 192.168.56.103
ns1.lab.local 192.168.56.101
lab.local 192.168.56.103
```

Con el resumen completo:

```
amass subs -show -d lab.local -ip
```

3.6 Ver todo lo que se le preguntó al DNS

> 📍 **Dónde:** en tu **portátil** (no dentro del contenedor)

Sal de la máquina `amass` primero (`exit`), y desde `lab-infra/docker`:

```
docker compose -f compose.yaml logs dns | grep "query" \
  | sed 's/.*\[query\] //' | awk '{print $1}' | sort -u
```

Aquí se ven los nombres que Amass consultó, **incluidos los que no existen**:

```
admin.lab.local
api.lab.local
dev.lab.local
grafana.lab.local
jenkins.lab.local
lab.local
mail.lab.local
ns1.lab.local
old.lab.local
qa.lab.local
staging.lab.local
test.lab.local      ← se preguntó, el DNS dijo que NO existe
vpn.lab.local
www.lab.local
```

> **Este es el comando clave de la demostración:** los candidatos aparecen porque se consultaron, pero solo quedan en los resultados los que el DNS confirmó.

3.7 Inspeccionar la base de datos de activos

> 📍 **Dónde:** dentro de la máquina `amass`
> 📍 **Herramienta:** `sqlite3` (ya viene instalada en la imagen)

Aquí es donde se ve el **grafo de activos** que mencionamos en la teoría. Cada subdominio es una **entidad** y cada relación entre activos es una **arista**.

3.7.1 Ver las tablas

```
sqlite3 /.config/amass/asset.db ".tables"
```

```
account           edge_tag       fqdn             location      productrelease
autnumrecord      edge_type_lu   fundstransfer    netblock      service
autonomoussystem  entity         gorp_migrations  organization  tag_type_lu
contactrecord     entity_tag     identifier       person        tlscertificate
domainrecord      entity_type_lu ipaddress        phone         url
edge              file           ipnetrecord      product
```

Las que importan para la demo:

Tabla	Qué guarda
`entity`	El índice de todos los activos (con su tipo y su clave natural)
`fqdn`	Los nombres de dominio: `www.lab.local`, `vpn.lab.local` ...
`ipaddress`	Las direcciones IP
`service`	Los servicios detectados en cada puerto
`edge`	**Las relaciones.** Es la tabla del grafo.
`entity_tag`	La **procedencia** de cada hallazgo: qué fuente lo descubrió y con qué confianza

3.7.2 Los nombres descubiertos

```
sqlite3 /.config/amass/asset.db "SELECT fqdn FROM fqdn ORDER BY fqdn;"
```

```
_sip._tcp.lab.local
admin.lab.local
api.lab.local
dev.lab.local
gitlab.lab.local
grafana.lab.local
jenkins.lab.local
lab.local
mail.lab.local
ns1.lab.local
qa.lab.local
stage.lab.local
staging.lab.local
test.lab.local
vpn.lab.local
www.lab.local
```

Fíjate en **`test.lab.local`** y **`stage.lab.local`**: están ahí. Amass los consultó, el DNS respondió que no existen, y aun así quedaron registrados como activos **con baja confianza**. Eso es el valor de la base de datos: no solo guarda lo que encontró, sino también lo que **descartó** y por qué.

3.7.3 Las direcciones IP

```
sqlite3 /.config/amass/asset.db "SELECT ip_address FROM ipaddress ORDER BY ip_address;"
```

```
192.168.56.101
192.168.56.103
192.168.56.104
192.168.56.105
192.168.56.106
192.168.56.107
192.168.56.108
192.168.56.109
```

3.7.4 El grafo: las relaciones entre activos

Esta es la consulta más importante. Une las tres tablas para mostrar **qué activo se relaciona con cuál y cómo**:

```
sqlite3 -header -column /.config/amass/asset.db "
SELECT f.fqdn AS origen, e.label AS relacion, ip.ip_address AS destino
FROM edge e
JOIN entity se ON se.entity_id = e.from_entity_id
JOIN fqdn f ON f.id = se.row_id
JOIN entity de ON de.entity_id = e.to_entity_id
LEFT JOIN ipaddress ip ON ip.id = de.row_id
ORDER BY f.fqdn LIMIT 12;"
```

```
     origen         relacion      destino
-----------------  ----------  -----------------
admin.lab.local    dns_record  192.168.56.105
api.lab.local      dns_record  192.168.56.109
dev.lab.local      dns_record  192.168.56.103
gitlab.lab.local   dns_record  192.168.56.107
grafana.lab.local  dns_record  192.168.56.108
jenkins.lab.local  dns_record  192.168.56.107
lab.local          dns_record  192.168.56.101
...
```

Qué significa cada columna:

Columna	Significado
`origen`	El activo desde el que sale la relación
`relacion`	El tipo de relación: `dns_record`, `node`, `tcp_port_443`...
`destino`	El activo al que apunta

`dns_record` significa "este nombre resuelve a esta IP". `node` significa "este subdominio cuelga de este dominio".

3.7.5 Cuántas relaciones se encontraron

```
sqlite3 /.config/amass/asset.db "SELECT COUNT(*) FROM edge;"
```

```
27
```

3.7.6 La procedencia: qué fuente descubrió cada activo

```
sqlite3 -header -column /.config/amass/asset.db "
SELECT f.fqdn AS activo, t.tag_type AS fuente, t.val AS valor
FROM entity_tag t
JOIN entity e ON e.entity_id = t.entity_id
JOIN fqdn f ON f.id = e.row_id
LIMIT 12;"
```

> La columna `tag_type` indica el origen del dato (`DNS`, `Known-FQDN`, etc.) y `val`, su nivel de confianza. Un activo descubierto por varias fuentes aparece con varias etiquetas.

3.7.7 Los logs del motor: cada relación en texto plano

Si lo de SQL te resulta ajeno, el motor también deja un log legible:

```
ls -t /.config/amass/amass_engine_*.log | head -1 | xargs grep "relationship discovered"
```

Cada línea es una relación descubierta, con el formato:

```
{"msg":"relationship discovered","from":"lab.local","relation":"dns_record","to":"192.168.56.103"}
```

3.7.8 Resumen: por qué esto importa

```
# Un resumen rápido de todo el grafo
sqlite3 /.config/amass/asset.db "
SELECT 'Activos FQDN: '   || COUNT(*) FROM fqdn
UNION ALL SELECT 'Direcciones IP: ' || COUNT(*) FROM ipaddress
UNION ALL SELECT 'Relaciones: '     || COUNT(*) FROM edge
UNION ALL SELECT 'Servicios: '      || COUNT(*) FROM service;"
```

Métrica	Por qué importa
Activos FQDN	El tamaño de la superficie descubierta
Direcciones IP	Cuánta infraestructura hay detrás
Relaciones	**La richness del grafo.** Si hay muchos activos y pocas relaciones, son nombres sueltos. Si hay muchas relaciones, hay un mapa.
Servicios	Qué está 실제로 expuesto en cada puerto

> **El argumento para el profesor:** un escáner que solo devuelve una lista de nombres te da N activos. Amass te da N activos **más** las relaciones entre ellos, la procedencia de cada uno y el historial de qué cambió. Eso es lo que convierte una lista de nombres en un inventario de superficie de ataque accionable.

3.8 Ver la zona DNS completa

> 📍 **Dónde:** dentro de la máquina `dns`

```
docker compose -f compose.yaml exec dns cat /app/zone.txt
```

O entrando directamente:

```
docker compose -f compose.yaml exec dns bash
cat /app/zone.txt
exit
```

Paso 4 · Experimentos para aprender

Experimento A — Pasivo vs. activo

> 📍 **Dónde:** dentro de la máquina `amass`

```
# Pasivo: no pregunta al DNS, solo consulta fuentes OSINT
amass enum -d lab.local -timeout 2

# Activo: pregunta al DNS y comprueba puertos
amass enum -active -d lab.local -p 80,443 -timeout 2
```

Qué observar: el pasivo termina rápido y sin descubrir nada. El activo tarda y encuentra subdominios.

Experimento B — Ampliar la lista de fuerza bruta

> 📍 **Dónde:** en tu **portátil**

```
cat >> lab-infra/wordlists/subdomains-lab.txt <<'EOF'
uat
preprod
sandbox
backup
EOF
```

> 📍 **Dónde:** dentro de la máquina `amass`

```
amass enum -engine http://127.0.0.1:4000 \
  -active -brute -w /wl/subdomains-lab.txt \
  -d lab.local -p 80,443 -timeout 3
```

Qué observar: `uat` y `preprod` existen en la zona. `prod` no existe:
puedes añadirlo y verificar que se consulta pero no aparece en los resultados.

Experimento C — Alteraciones de nombres

> 📍 **Dónde:** dentro de la máquina `amass`

```
amass enum -engine http://127.0.0.1:4000 \
  -active -alts -aw /wl/alterations-lab.txt \
  -d lab.local -p 80,443 -timeout 2
```

Qué observar: a partir de `dev.lab.local` genera variantes (`dev1` ,
`dev-test` , `test-dev` ...) y valida cada una contra el DNS.

Experimento D — Cambiar el alcance de puertos

> 📍 **Dónde:** dentro de la máquina `amass`

```
amass enum -engine http://127.0.0.1:4000 \
  -active -d lab.local -p 80,443,8080 -timeout 2
```

Experimento E — Comparar con el scope

> 📍 **Dónde:** dentro de la máquina `amass`

```
amass subs -names -d lab.local -ip
amass subs -names -d noexiste.lab.local -ip
```

Paso 5 · Resumen: dónde se ejecuta cada cosa

Acción	Dónde	Comando
Montar el laboratorio	**Portátil**	`./start.sh`
Empezar de cero	**Portátil**	`./start.sh --reset`
Entrar a la máquina Amass	**Portátil**	`cd lab-infra/docker && docker compose -f compose.yaml exec amass bash`
Salir de la máquina	**Dentro**	`exit`
Ver versión	**Amass**	`amass -version`
Ayuda de un subcomando	**Amass**	`amass enum -h`
Consultar el DNS	**Amass**	`dig +short @192.168.56.101 dev.lab.local A`
Enumerar + fuerza bruta	**Amass**	`amass enum -engine http://127.0.0.1:4000 -active -brute -w /wl/subdomains`

		`-lab.txt -d lab.local -p 80,443 -timeout 3`
Ver resultados	**Amass**	`amass subs -names -d lab.local -ip`
Ver la base de datos	**Amass**	`sqlite3 /.config/amass/asset.db "SELECT fqdn FROM fqdn;"`
Ver consultas DNS	**Portátil**	`docker compose -f compose.yaml logs dns | grep query`
Ver la zona	**DNS**	`docker compose -f compose.yaml exec dns cat /app/zone.txt`
Detener todo	**Portátil**	`./start.sh --stop`

Los atajos del script

Si no quieres escribir comandos de Docker, `start.sh` los hace por ti:

Comando	Qué hace
`./start.sh`	Monta el laboratorio y verifica el DNS
`./start.sh --reset`	Empieza de cero
`./start.sh --enum`	Ejecuta la enumeración con fuerza bruta
`./start.sh --results`	Muestra los resultados guardados
`./start.sh --dns`	Muestra las consultas DNS
`./start.sh --check`	Verifica el estado
`./start.sh --logs`	Logs en vivo
`./start.sh --stop`	Detiene el laboratorio
`./start.sh amass <args>`	Ejecuta un comando de Amass a mano

Ejemplo:

```
./start.sh amass subs -names -d lab.local -ip
```

La zona del laboratorio

`lab.local` tiene 32 nombres:

Grupo	Nombres	IP
Objetivo	`www`, `mail`, `www2`	192.168.56.103
Preproducción	`dev`, `staging`	192.168.56.103
Preproducción	`qa`, `uat`, `preprod`	192.168.56.104
Administración	`admin`, `portal`	192.168.56.105
Acceso remoto	`vpn`	192.168.56.106
CI/CD	`jenkins`, `gitlab`	192.168.56.107
Observabilidad	`grafana`, `kibana`	192.168.56.108
APIs	`api`, `beta`, `sandbox`	192.168.56.109
Residuales	`old`, `backup`	192.168.56.110
Infraestructura	`ns1`	192.168.56.101
Infraestructura	`sipserver` (+ SRV `_sip._tcp`)	192.168.56.111
Aliases CNAME	`intranet` → `portal`, `secure` → `vpn`, `ci` → `jenkins`, `scm` → `gitlab`, `metrics` → `grafana`, `logs` → `kibana`, `preview` → `beta`, `legacy` → `old`	—

Priorización

Activo	Severidad	Razón
`vpn.lab.local`	**Crítico**	Acceso remoto a toda la organización
`dev` / `staging` / `qa` / `uat`	**Alto**	Preproducción con menos controles
`jenkins` / `gitlab`	**Alto**	Cadena de suministro
`admin` / `portal`	**Alto**	Administración expuesta
`old` / `backup`	**Medio**	Activo residual
`www` / `mail`	**Bajo**	Activos públicos esperados

> **La conclusión:** Amass **no prioriza**. Entrega el inventario con su procedencia. La severidad depende del contexto: si una IP es producción o una máquina de pruebas, el mismo hallazgo cambia. Eso lo decide el equipo, no el script.

Problemas frecuentes

Síntoma	Causa	Solución
`Failed to perform a bulk transfer of assets: EOF`	Bug de Amass v5.1.3 en el endpoint `bulk`	No uses `-nf`. Usa `-brute -w`.
El motor muere con `panic: invalid memory address`	Mismo bug	`./start.sh --reset` y usa `-brute -w`
`Failed to create a session with the Amass engine`	El motor no está listo	`./start.sh` espera al motor. Si persiste, `--reset`.
`Failed to obtain the BGPTools IP address`	`bgp.tools` no resuelve	Verifica que está en `lab-infra/docker/zone.txt` y haz `--reset`
`flag provided but not defined: -src` / `-ip` / `-o`	Banderas de v3/v4 que ya no existen	Usa `amass subs` (ver §3.5)
`subcommand provided but not defined: db`	El subcomando `db` fue eliminado	Usa `amass subs` o `sqlite3` (§3.7)
La enumeración no descubre nada	Faltan las `transformations` en el config	Restaura el bloque `transformations:` de `lab-infra/configs/config.yaml`
`service "amass" is not running`	Los contenedores están detenidos	`./start.sh` los levanta
`Address already in use`	Ya hay contenedores con esos nombres	`./start.sh --stop` y vuelve a levantar
`permission denied` en iptables	Falta `NET_ADMIN`	Está en el `compose.yaml`; no lo borres
`Failed to populate the ASN cache: zero entities found`	Normal: requiere WHOIS/RDAP	Ignorar
La enumeración tarda mucho	El plugin DNS-SD hace ~60 consultas SRV	Reduce `-timeout` a 2
`docker: permission denied`	Tu usuario no está en el grupo `docker`	`sudo usermod -aG docker $USER` y reinicia sesión

---

Dos detalles técnicos que vale la pena conocer

1. `transformations` no es opcional

En Amass v5 el motor carga esa sección del `config.yaml` al leer el archivo. Si falta, **ningún plugin se activa** y la enumeración termina sin descubrir nada, sin error visible. Por eso `lab-infra/configs/config.yaml` la incluye completa.

2. Los resolvers DNS están escritos en el código

El plugin DNS de v5 usa una lista fija de 53 servidores DNS públicos. La opción `resolvers:` del config solo aplica a `intel` / `whois`. Para que las consultas lleguen al DNS ficticio, la imagen de Amass trae reglas de redirección preconfiguradas que se aplican solas al arrancar. No hay que hacer nada.

Referencias

- Sitio del proyecto: https://owasp.org/projects/amass
- Documentación oficial v5: https://owasp-amass.github.io/docs/
- Código fuente: https://github.com/owasp-amass/amass

Guía validada con OWASP Amass v5.1.3 sobre Docker.